Synapse

Dossier de développement : spécification produit et design

Version 1.0 · 27 octobre 2026

Ce document accompagne les maquettes interactives du canevas « Synapse – maquettes ». Les maquettes font foi pour le visuel ; ce document fait foi pour les règles de fonctionnement. Les choix techniques retenus pour la version 1 figurent en dernière section.

1. Le produit

Synapse est une plateforme où une équipe d'agents IA conçoit, développe, teste, documente et déploie des applications. Chaque agent tient un rôle précis d'une équipe logicielle (une « casquette »). L'humain ne fait plus le travail lui-même : il lance les demandes, relit ce que l'équipe propose et décide aux étapes clés.

La règle qui guide tout le reste : **l'IA propose, l'humain valide**. L'exécution, les contrôles et l'amélioration continue sont automatiques ; les décisions restent humaines.

Principes non négociables

- Aucune étape clé ne passe sans validation humaine : exigences, maquettes, architecture, plan de chaque tâche, résultat de chaque tâche, mise en production.
- Chaque tâche reçoit deux validations : le plan avant l'exécution, le résultat après.
- Si un agent doit s'écarter d'un plan validé, il s'arrête et redemande une validation. Il ne change jamais de plan en silence.
- La QA est indépendante du développement : elle construit ses tests à partir des exigences validées, jamais à partir du code.
- Un bug est toujours reproduit par un test qui échoue avant d'être corrigé. Ce test rejoint ensuite la suite de non-régression.
- La mise en production n'est jamais automatique.
- Aucun agent ne manipule de secret en clair. Les secrets restent dans un coffre lu directement par la pipeline.
- Chaque proposition affiche ce dont l'agent n'est pas sûr, ce qu'il a supposé et ce qu'il n'a pas pu vérifier.

2. Utilisateurs

Rôle	Ce qu'il fait dans Synapse	Ce qu'il voit
Product Owner	Crée ou importe des projets, lance les demandes (Feature, Bugfix, Doc), valide plans et résultats, priorise le backlog, choisit le modèle IA, déclenche la mise en production.	Tous ses projets, toutes les tâches, la mémoire et les environnements.
Testeur	Teste sur Staging, signale les bugs avec un formulaire structuré, confirme les corrections.	Uniquement ses bugs, classés par statut.
Administrateur d'espace	Invite les membres, configure le SSO, connecte les fournisseurs Git, choisit les modèles IA disponibles.	Paramètres de l'espace. En version 1, gestion dans la console Keycloak (section 12).

3. Les casquettes

L’orchestrateur choisit automatiquement la casquette qui agit selon le type de tâche et l’étape. L’utilisateur ne choisit jamais la casquette, seulement le modèle IA.

Casquette	Livrable	Référentiel métier	Outils
Orchestrateur	Répartition du travail, passage de relais, statuts des tâches	Règles de cycle de vie (section 5)	File de tâches, mémoire du projet
Business Analyst	Exigences, règles métier, critères d’acceptation, questions au PO	Ingénierie des exigences, glossaire métier du projet	Lecture des documents joints
UX/UI Designer	Parcours, maquettes d’écrans	WCAG 2.2 AA, design system du projet	Outil de maquettage
Architecte	Architecture, modèle de données, découpage, analyse d’impact	Conventions de la stack, décisions d’architecture passées	Lecture du dépôt
Développeur	Code front et back	Conventions de code du projet	Dépôt, exécution en bac à sable
Reviewer de code	Revue avec corrections demandées ou approbation	Checklist de revue	Diff, analyse statique
Analyste de test	Scénarios écrits dans le modèle Excel du projet	ISTQB CTFL v4 : partitions, valeurs limites, tables de décision, transitions d’état	Modèle Excel
Automaticien	Choix du scope E2E par le risque, tests automatisés	Pyramide de tests, bonnes pratiques Playwright	Playwright, exécution en bac à sable
Expert sécurité	Rapport de vulnérabilités	OWASP Top 10, OWASP ASVS	Analyse statique, scan des dépendances
Débogueur	Reproduction, cause, correctif	Procédure de reproduction	Exécution des tests
Rédacteur technique	Documentation utilisateur et technique	Guide de style du projet	Mémoire du projet
DevOps	Pipelines, déploiements, retours arrière	Infrastructure as code, CI/CD, contrôles de santé	Pipeline, coffre de secrets

Ce qui rend une casquette experte

- **Un référentiel versionné**, chargé automatiquement avant chaque travail. L’agent peut proposer de l’enrichir à partir des corrections du PO ; seul un humain accepte une nouvelle règle.
- **Une définition de « terminé »** vérifiée par l’agent avant de rendre son livrable.
- **Une relecture par une autre casquette** avant que le livrable n’arrive devant l’humain.
- **Des cas de référence** rejoués automatiquement à chaque changement de modèle ou de prompt. L’humain n’est alerté que si une casquette régresse.

4. Projets : créer ou importer

Créer de zéro

Le PO donne un nom, décrit l'application et joint ses documents (cahier des charges, modèle Excel de scénarios). Pipeline :

Étape	Qui	Validation humaine
Exigences	Business Analyst	Oui
Maquettes	UX/UI Designer	Oui
Architecture	Architecte	Oui. Le dépôt et les environnements sont créés à ce moment.
Code, revue, tests, sécurité, documentation	Développeur, Reviewer, Analyste de test, Automaticien, Expert sécurité, Rédacteur	Recette du résultat
Déploiements	DevOps	Mise en production

Importer un projet existant

Le PO connecte un fournisseur Git (GitHub, GitLab, Bitbucket) ou colle l'URL du dépôt, choisit la branche, décrit où en est le projet et joint les documents existants. Synapse étudie le projet puis reprend là où il s'est arrêté.

Étape	Qui	Ce qui est produit
Étude du code	Architecte	Stack, taille, modules, dette technique, failles connues des dépendances
Tests existants	Analyste de test	Inventaire des tests, couverture, absence d'E2E
Exigences reconstituées	Business Analyst	Exigences déduites du code et des documents, écarts entre spécification et code
Validation de la mémoire	PO	Le PO tranche chaque écart (garder la spécification ou garder le code)
Reprise du backlog	Orchestrateur	TODO du code et issues ouvertes, importés en Backlog avec leur origine
Priorisation	PO	Le cycle normal des tâches reprend

5. Tâches

Types

Type	Préfixe	Chaîne de casquettes
Feature	FEAT-	BA (analyse d’impact) → Architecte (plan) → Développeur → Reviewer → QA et sécurité → Rédacteur si besoin
Bugfix	BUG-	Débogueur (reproduction et test en échec) → plan de correction → correction → test au vert et non-régression → confirmation du testeur
Doc	DOC-	Rédacteur technique → relecture

Statuts et transitions

Statut	Ce qui s’y passe	Pour en sortir
Backlog	Demandes créées, bugs signalés, tâches importées	Le PO priorise : passage en To do
To do	L’agent prépare son plan	Le PO valide le plan : passage en In progress
In progress	L’IA exécute, les contrôles automatiques tournent	Livrable contrôlé : passage en Review. Question ou écart au plan : la tâche reste en In progress, marquée « en attente de réponse », et apparaît dans les validations du PO
Review	Le résultat attend la relecture humaine	Validé : passage en Done (pour un Bugfix, après confirmation du testeur). Refusé : retour en In progress avec les commentaires
Done	Tâche terminée, livrables versés dans la mémoire	—

- La boucle QA et développement s’arrête après 3 essais ; au-delà, la tâche remonte au PO.
- Tout l’historique des allers-retours est conservé : propositions, refus, commentaires, versions.

6. Environnements et déploiement

Environnement	Déploiement	Rôle
Staging	Automatique après revue et tests au vert	Les testeurs y travaillent ; les bugs signalés portent sur une version connue
Préprod	Automatique après validation du résultat par le PO	E2E, non-régression et scan de sécurité rejoués sur cet environnement
Prod	Bouton humain uniquement	Le DevOps prépare le contenu, l’état des contrôles et le plan de retour arrière

- Retour arrière automatique vers la version précédente si un contrôle de santé échoue après un déploiement.
- Journal du DevOps : chaque déploiement, retour arrière et mise en production, avec son auteur (« Auto » ou nom de l’humain).

7. Mémoire du projet

La mémoire regroupe tout ce que l’humain a validé. Elle est versionnée dans le dépôt, à côté du code, et relue automatiquement par l’agent avant chaque nouvelle tâche.

Document	Contenu
Exigences	Exigences, règles métier, critères d’acceptation
Maquettes	Parcours et écrans validés
Architecture	Choix techniques, modèle de données, découpage
Scénarios	Scénarios de test au format du modèle Excel (section 12), avec un statut d’exécution par plateforme ; export Excel
Tests E2E	Parcours critiques automatisés et justification de leur choix
Décisions	Réponses du PO aux questions des agents, datées

8. Modèles IA

Le modèle qui fait tourner les agents se choisit dans la barre de saisie : Claude Opus, Claude Sonnet, GPT, Gemini, Mistral ou un modèle local hébergé sur l’infrastructure du client. La liste exacte dépend de ce que l’administrateur active. Le code passe par une passerelle unique (section 12) pour changer de fournisseur sans toucher aux casquettes, et rejouer les cas de référence à chaque changement.

9. Écrans

#	Écran	Contenu et interactions
0	Design system	Logo, couleurs, typographie, statuts, types, boutons, composants
1	Connexion	SSO entreprise ou e-mail et mot de passe
2	Accueil	Message d’accueil, décisions en attente sur tous les projets (plans, résultats, mises en production), cartes de projets avec avancement par statut
3	Nouveau projet	Bascule « Créer de zéro » / « Importer un projet existant » ; formulaire et pipeline adaptés au choix
3b	Import	Fil de l’étude : état du code, mémoire reconstituée, écarts à trancher, validation de la mémoire
4	Fil Feature	Conversation avec les casquettes, frise des casquettes, carte de plan à valider ; la tâche change de statut à la validation
5	Fil Bugfix	Signalement du testeur, reproduction, correction ; panneau droit avec le diff et les tests ; validation du résultat
6	Testeur	Ses bugs par statut, formulaire de signalement (titre, environnement, appareil, étapes, attendu, obtenu, gravité, captures)
7	Mémoire du projet	Six documents validés, aperçu des scénarios au format Excel, export
8	Environnements	Chaîne Staging, Préprod, Prod ; carte de mise en production ; journal du DevOps

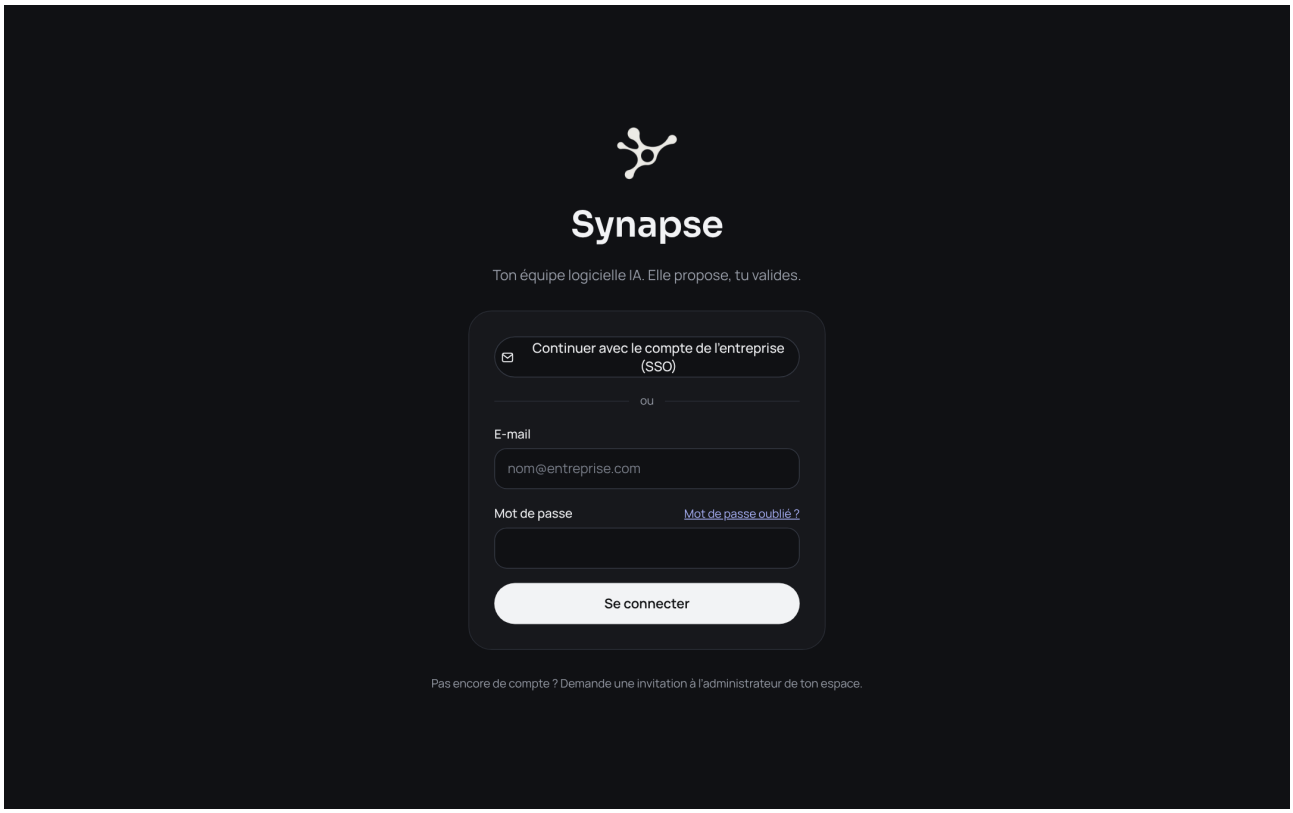
Structure commune des écrans de projet

- **Barre latérale** : logo Synapse (retour à l’accueil), nom du projet avec sélecteur, bouton nouvelle tâche, tâches classées par statut dans l’ordre Backlog, To do, In progress, Review, Done, puis le profil de la personne connectée en bas.
- **Zone centrale** : le fil de conversation. Messages humains en bulle à droite, réponses des casquettes en texte libre avec le nom de la casquette, propositions sous forme de cartes avec leurs boutons de décision.
- **Barre de saisie** : pièce jointe, dictée, type de tâche (Feature, Bugfix, Doc), sélecteur de modèle IA, mention « Validation manuelle ».
- **Panneau droit** : s’ouvre sur un livrable (diff, tests, documents).

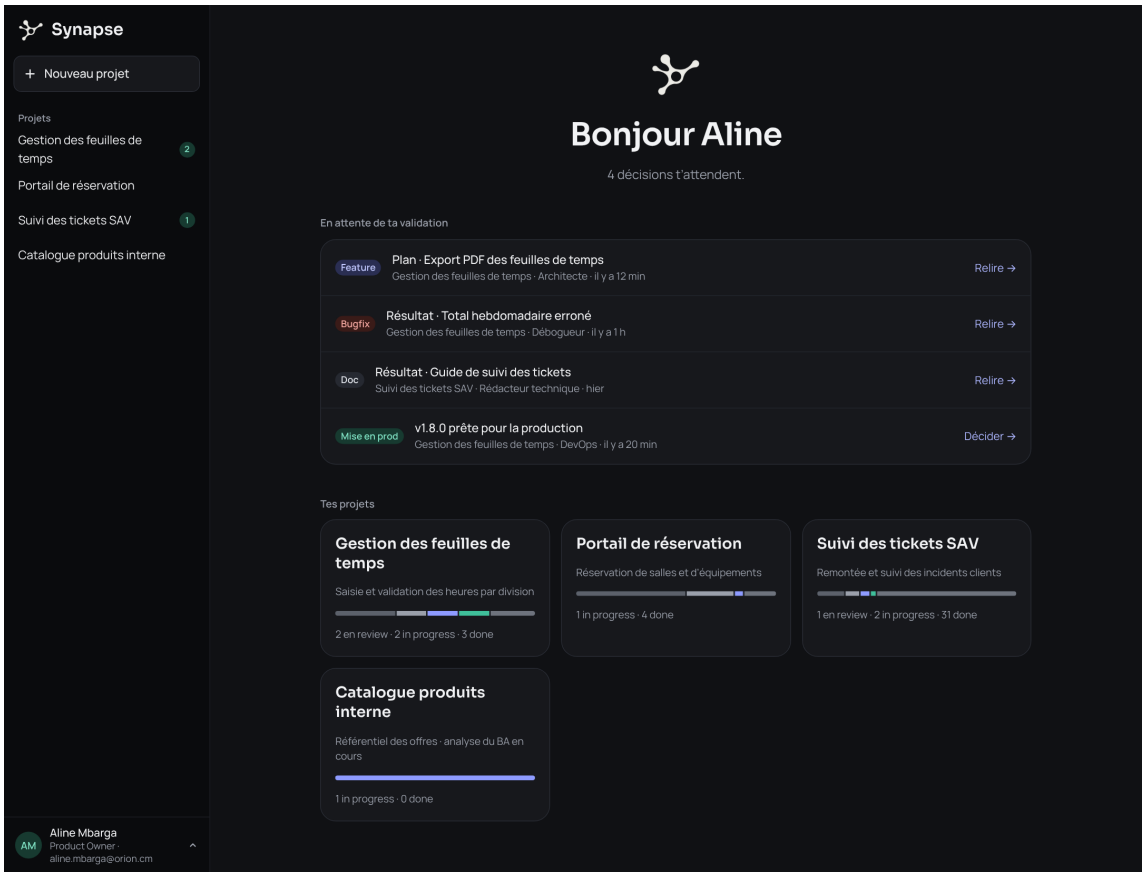
Maquettes

Captures des écrans à 1440 px de large, dans leur état initial. Les états après action (plan validé, mise en production, bug envoyé) sont visibles dans le prototype interactif.

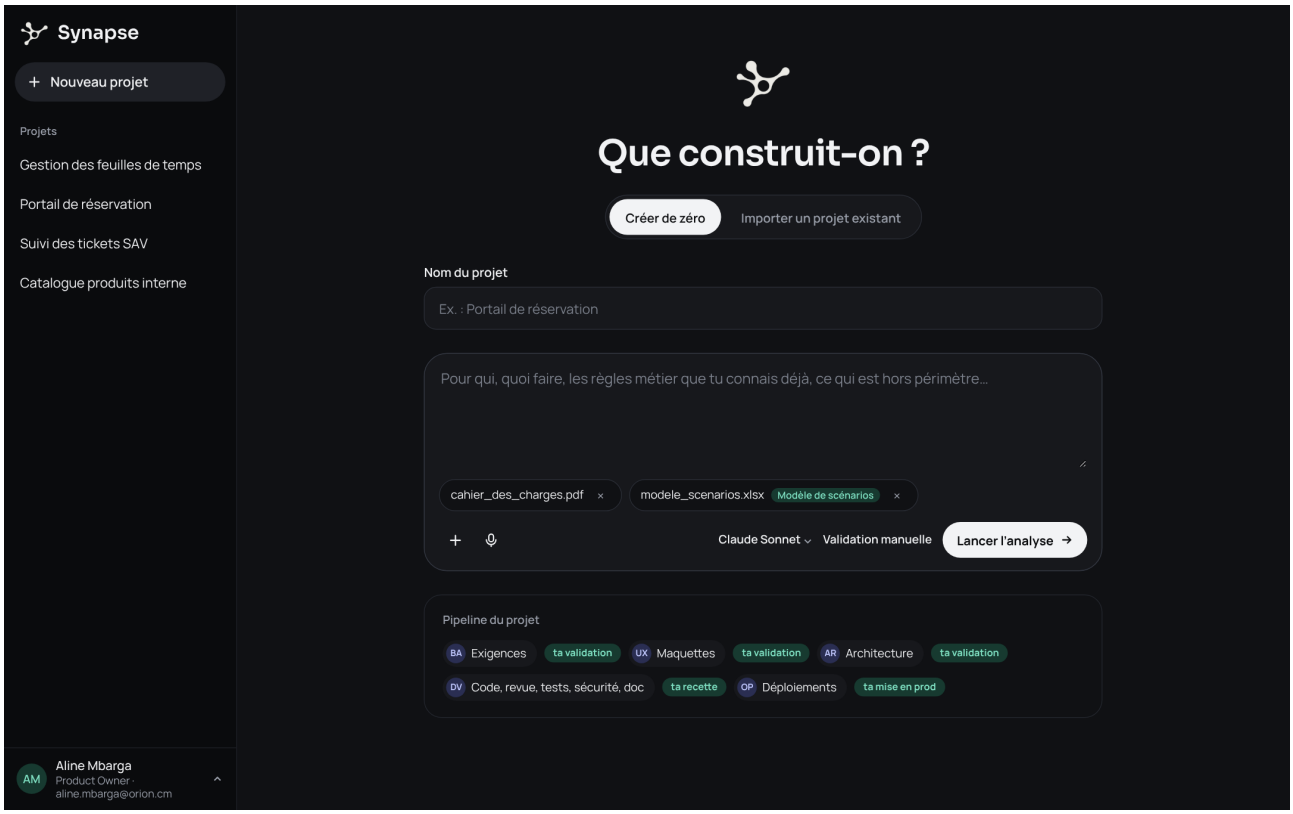
1 · Connexion



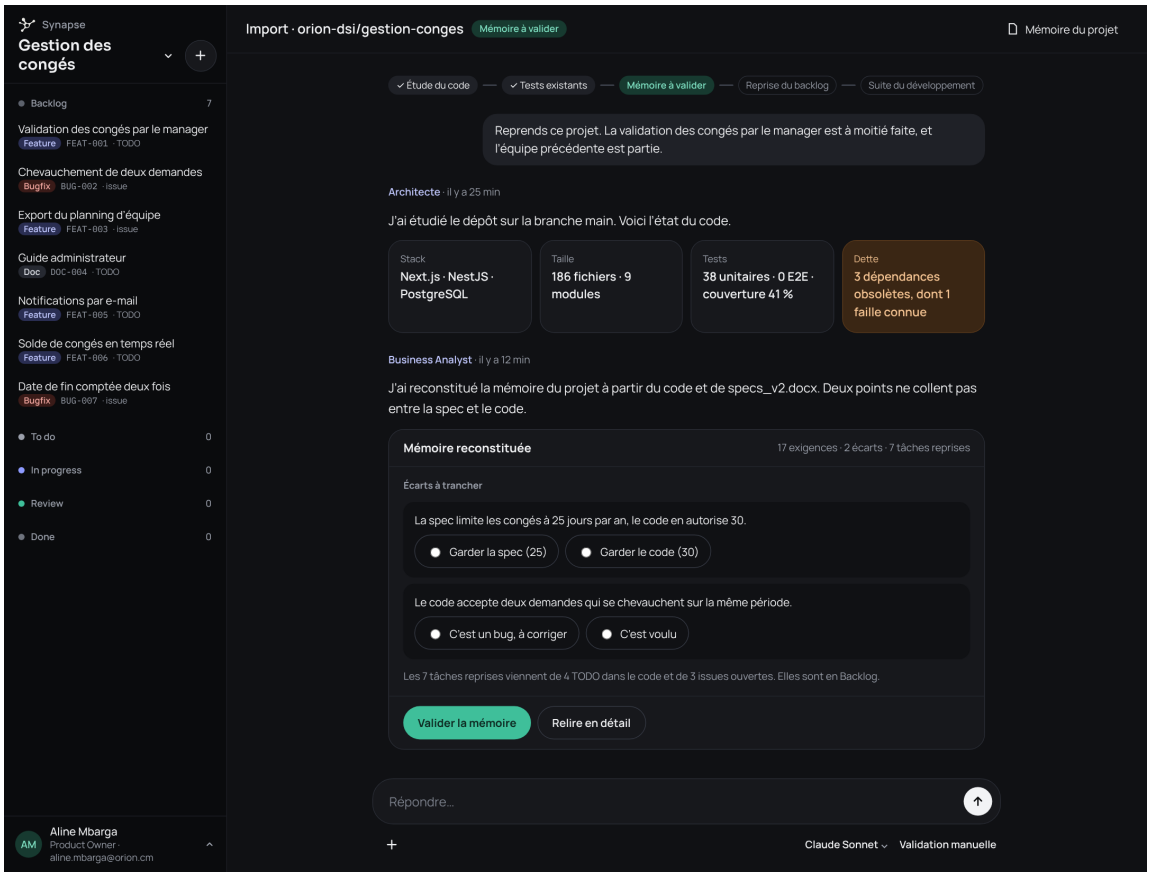
2 · Accueil



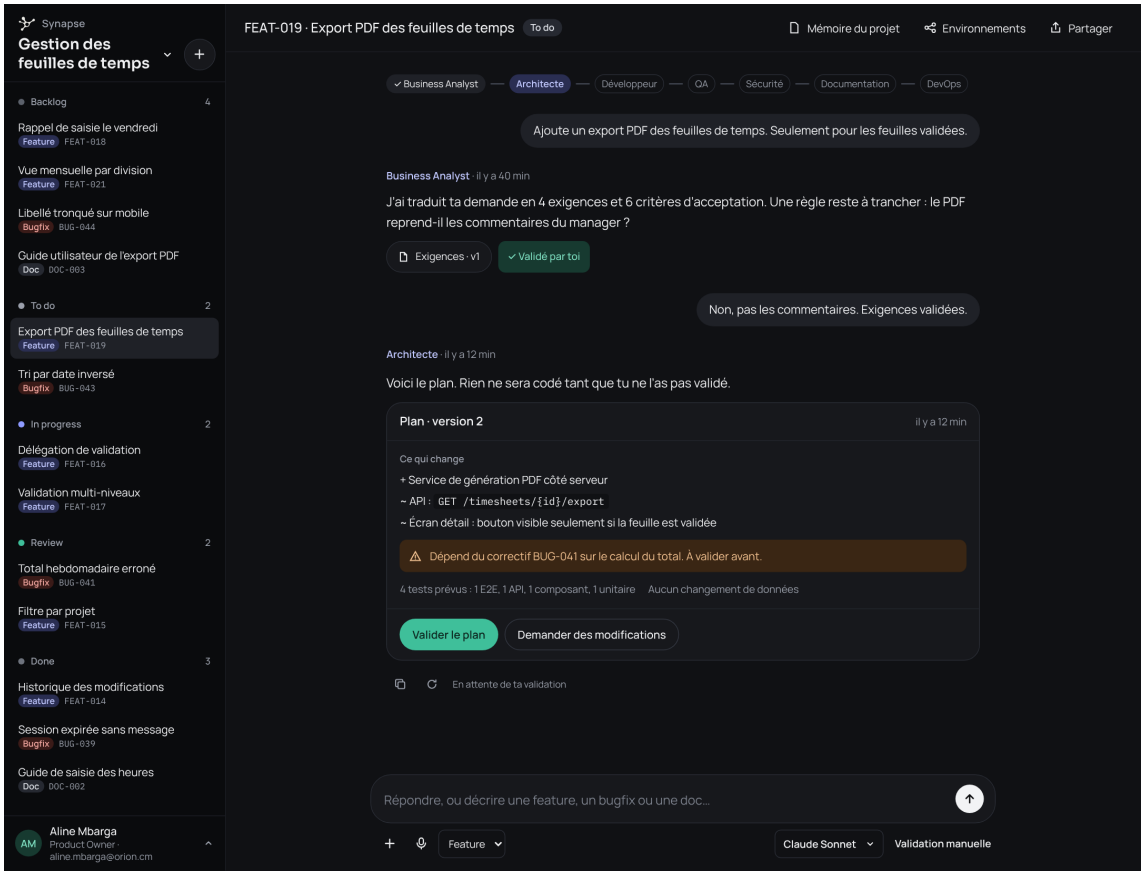
3 · Nouveau projet



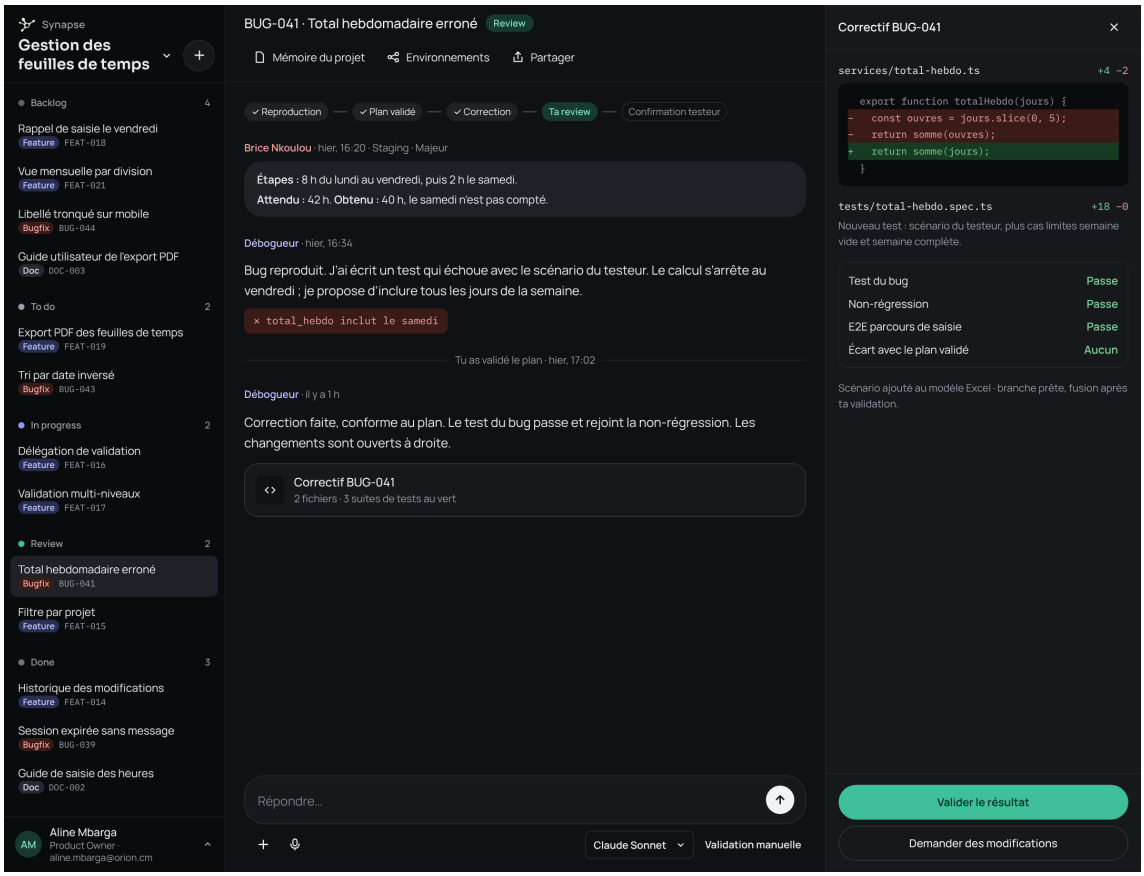
3b · Import d’un projet existant



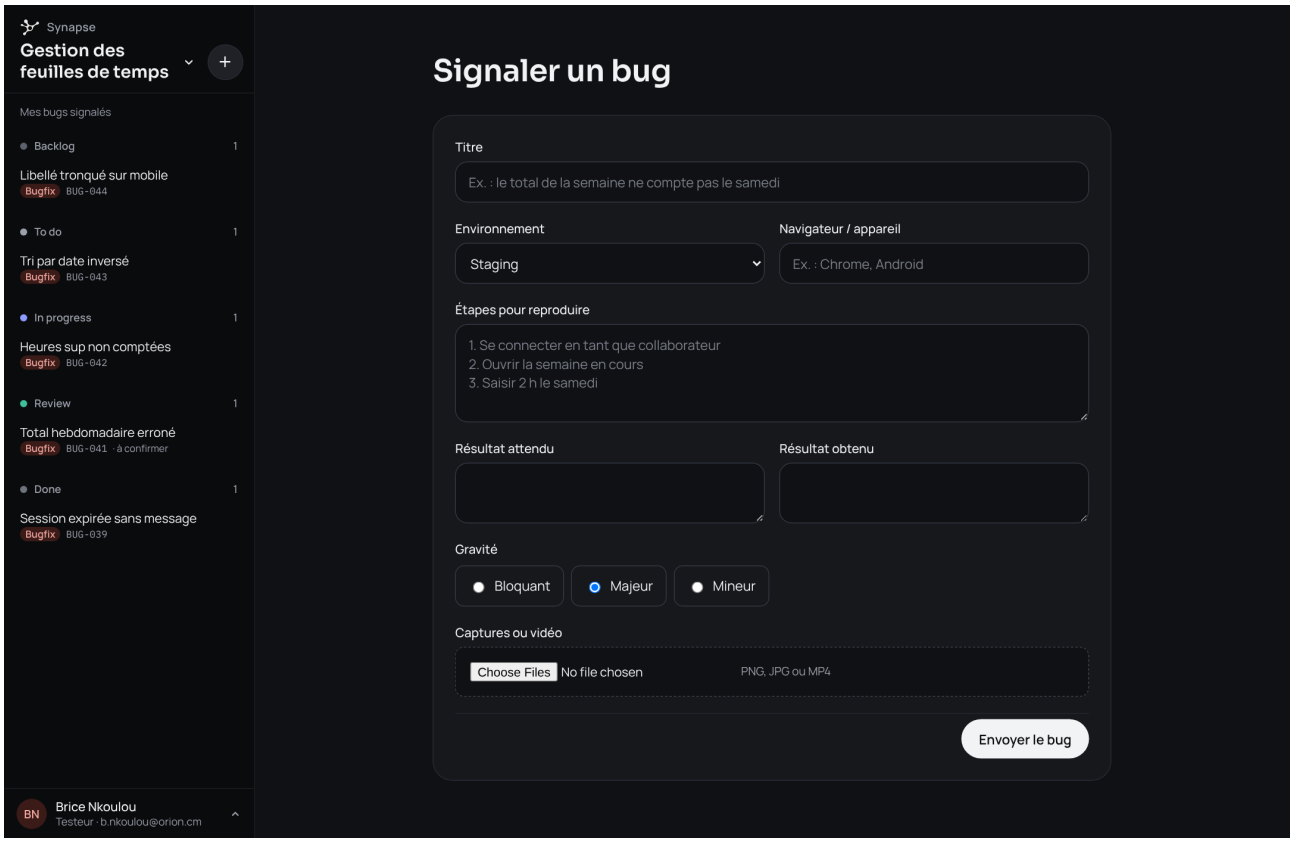
4 · Fil Feature, plan à valider



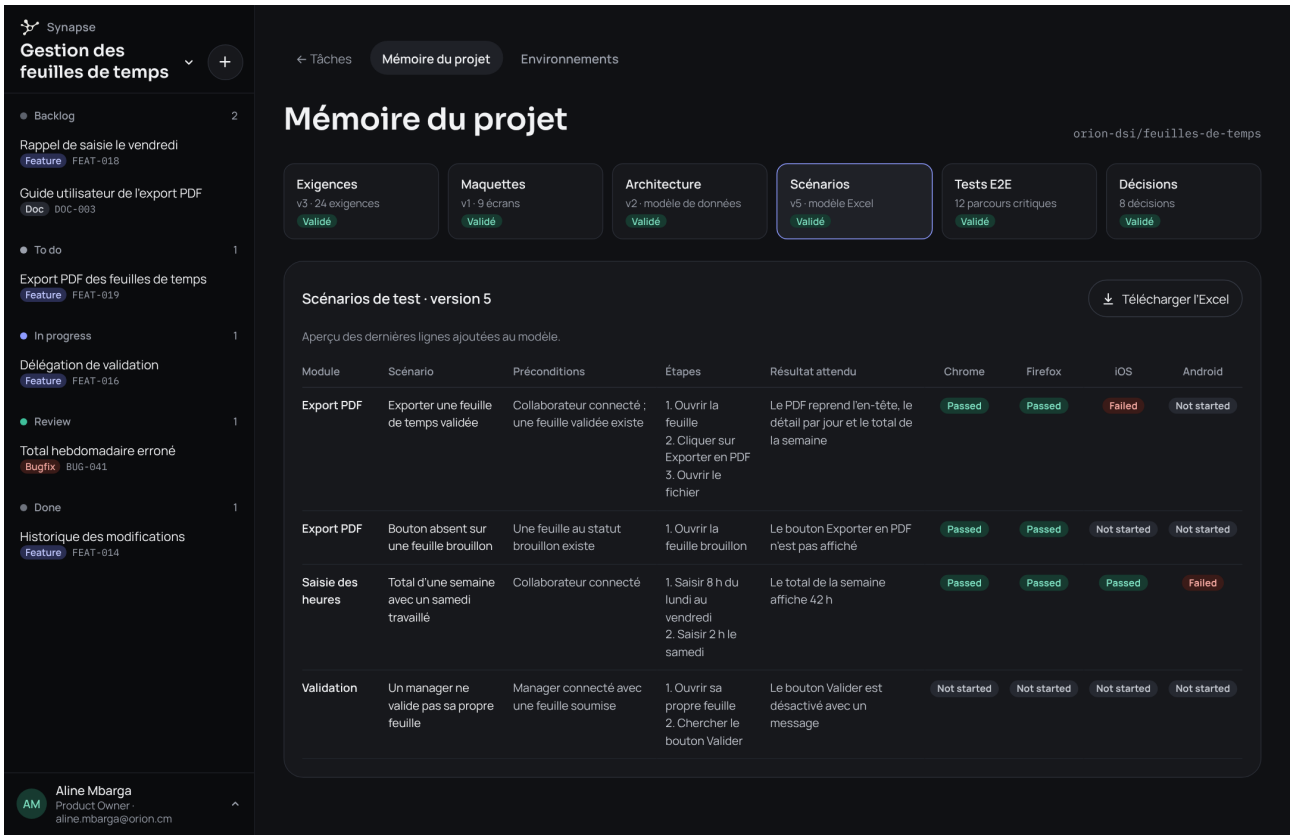
5 · Fil Bugfix, résultat en review



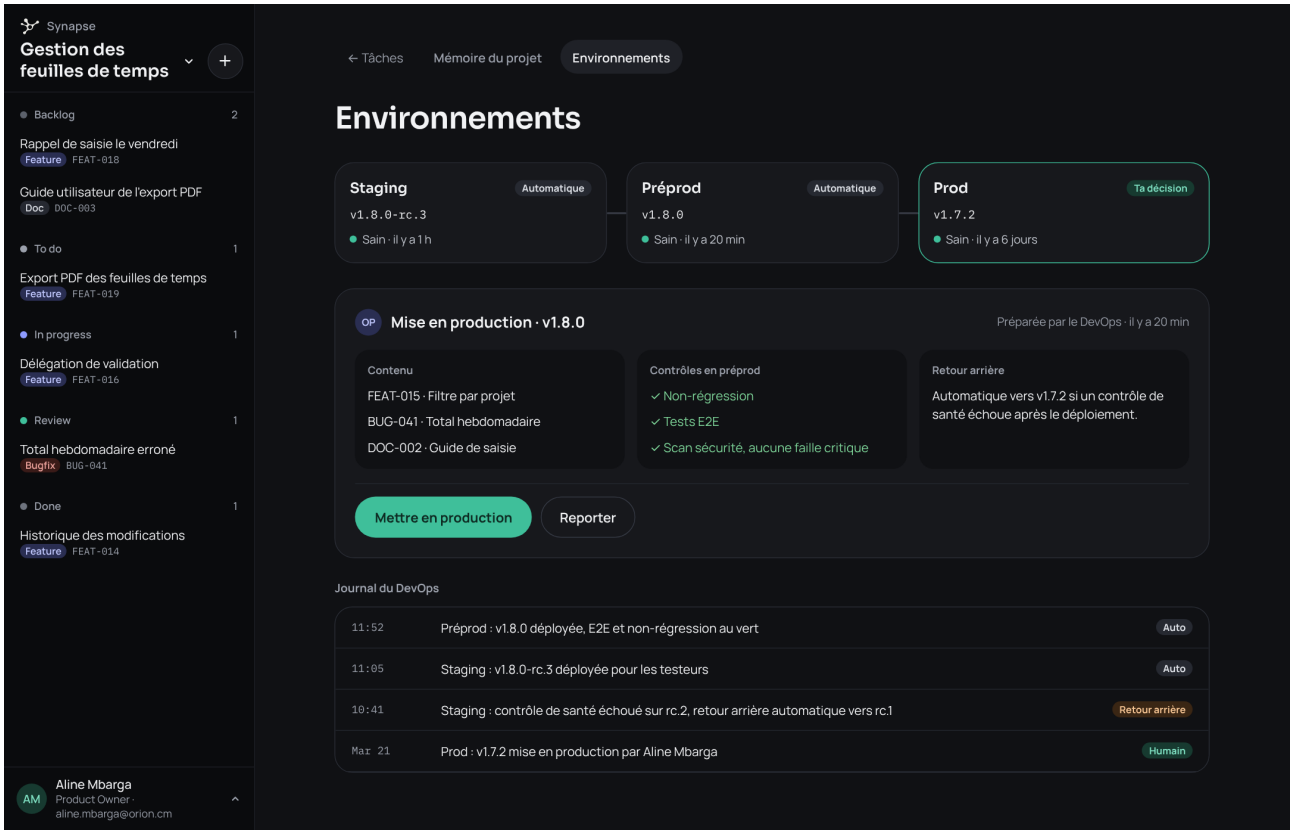
6 · Testeur, signaler un bug



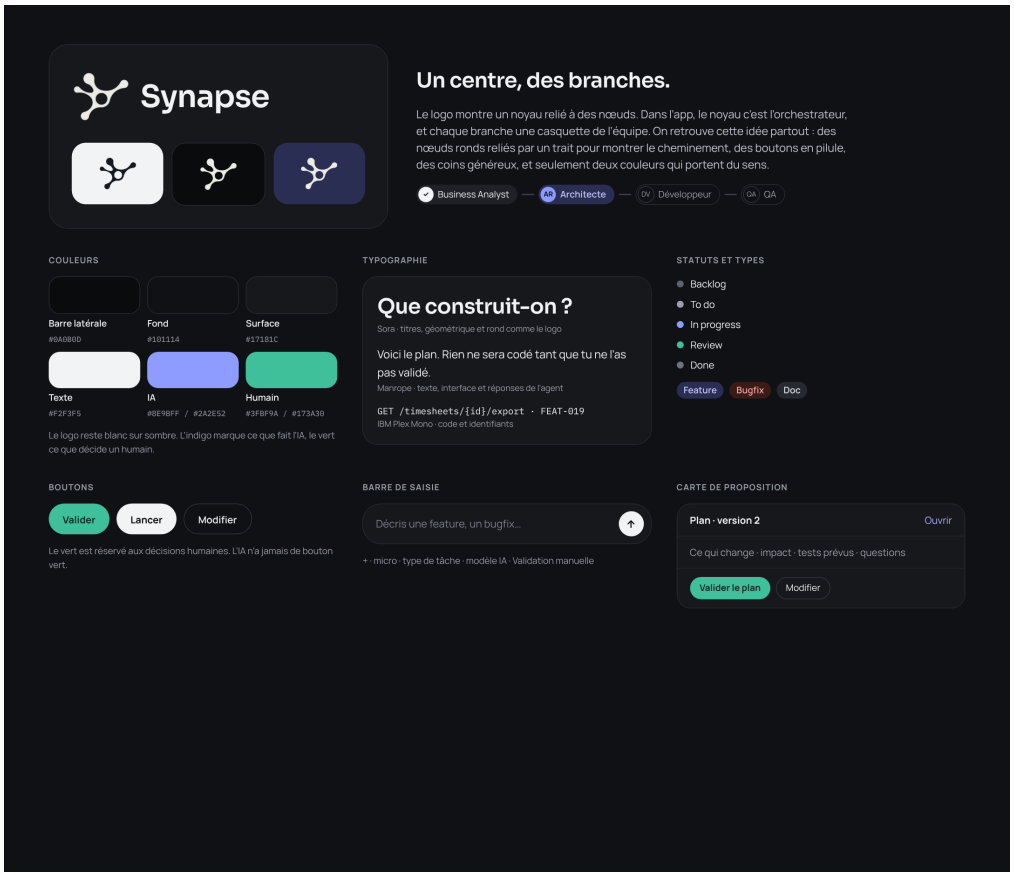
7 · Mémoire du projet



8 · Environnements et mise en production



0 · Design system



10. Design system

L’identité découle du logo : un noyau relié à des branches. Le noyau est l’orchestrateur, les branches sont les casquettes. On retrouve des nœuds ronds reliés par un trait, des boutons en pilule et des coins généreux.

Jeton	Valeur	Usage
Barre latérale	#0A0B0D	Fond de la barre latérale
Fond	#101114	Fond de l’application
Surface	#17181C	Cartes, barre de saisie
Surface active	#1F2127	Élément sélectionné, bulle humaine
Bordure	#2A2D35	Contours des cartes et champs
Texte	#F2F3F5	Texte principal
Texte secondaire	#9BA0AB	Métadonnées, libellés
IA	#8E9BFF sur #2A2E52 (texte #C2C9FF)	Tout ce que fait l’IA, statut In progress, type Feature
Humain	#3FBF9A sur #173A30 (texte #7FE0C2)	Décisions humaines uniquement, statut Review
Bugfix	#F2A7A0 sur #3A1B18	Type Bugfix, erreurs
Attention	#F5C493 sur #3A2614	Impacts, dette, retours arrière
Statuts	Backlog #5C616B · To do #9BA0AB · In progress #8E9BFF · Review #3FBF9A · Done #6E737D	Pastilles de statut

Élément	Règle
Typographie	Sora (titres), Manrope (interface et réponses des agents), IBM Plex Mono (code et identifiants)
Rayons	Boutons et étiquettes en pilule ; cartes 20 à 22 px ; barre de saisie 26 px ; champs 14 px ; avatars ronds
Boutons	Vert plein : décision humaine (Valider, Mettre en production). Clair plein : action principale (Lancer, Envoyer). Contour : action secondaire
Logo	Version claire sur fond sombre, version sombre sur fond clair (fichiers PNG fournis)
Accessibilité	Contrastes WCAG AA, cibles tactiles de 44 px minimum, vrais boutons et libellés de champs, rendu correct sur mobile

11. Exigences techniques transverses

- **Traçabilité** : chaque proposition et chaque décision est journalisée avec l’auteur, la date, la casquette, le modèle IA et la version du référentiel utilisés.
- **Sécurité** : SSO entreprise, droits par rôle, exécution du code généré dans un bac à sable isolé, secrets en coffre, aucun secret dans les messages ou la mémoire.
- **Évaluation continue** : cas de référence par casquette, rejoués automatiquement ; tableau de bord des régressions pour l’administrateur.
- **Coûts** : suivi de la consommation par projet et par modèle ; quotas configurables par l’administrateur.
- **Robustesse** : limite d’essais sur les boucles automatiques, reprise d’une tâche interrompue sans perte de contexte.

12. Choix techniques

Décisions retenues pour la première version.

Sujet	Choix	Pourquoi
Front	Next.js en TypeScript	Adapté à l’interface des maquettes ; technologie déjà maîtrisée dans l’équipe
Back	Python, FastAPI	L’écosystème des agents IA est d’abord disponible en Python
Données	PostgreSQL avec l’extension pgvector	Projets, tâches, historique et mémoire ; recherche rapide dans la mémoire des gros projets
Fichiers	Stockage objet compatible S3, auto-hébergé	Modèles Excel, captures, documents joints
Orchestration	LangGraph, état sauvegardé dans PostgreSQL	Compatible avec plusieurs fournisseurs de modèles, pauses en attente de validation humaine, reprise après interruption
Casquettes	Un sous-graphe LangGraph par casquette	Chaque casquette porte son référentiel, sa checklist « terminé » et ses outils
Exécution du code	Conteneur isolé, un par tâche	Le code généré ne s’exécute jamais sur les serveurs de Synapse
Accès aux modèles	Passerelle unique de type LiteLLM	Changer de modèle sans toucher aux casquettes ; suivi des coûts par projet
Évolution prévue	Temporal, via son plugin LangGraph	Seulement si les attentes de validation longues et le volume l’exigent ; pas en version 1
Hébergement	Cloud privé, cluster Kubernetes léger	Coût maîtrisé, souveraineté des données ; modèles appelés par API au démarrage, modèle local quand des GPU seront disponibles
Identité	Keycloak auto-hébergé (OpenID Connect)	Connexion, rôles PO, Testeur et Admin, SSO vers les comptes Microsoft ou Google des clients
Git	GitLab en premier, GitHub ensuite	GitLab s’auto-héberge et sa CI/CD sert directement au DevOps ; GitHub pour les clients qui y sont déjà
Tickets	Aucun outil externe en version 1	Le backlog vit dans Synapse

Règles chiffrées

Règle	Valeur
Allers-retours QA et débogueur avant remontée au PO	3
Retour arrière automatique après une mise en production	Actif pendant 30 minutes
Durée maximale d’exécution d’une tâche par un agent	2 heures, puis remontée au PO

Modèle Excel des scénarios

L’analyste de test écrit chaque scénario sur une ligne, dans cet ordre de colonnes :

Colonne	Contenu
Module	Partie de l’application concernée

Colonne	Contenu
Scénario	Ce qui est testé, en une phrase
Préconditions	État nécessaire avant de commencer (utilisateur connecté, données existantes…)
Étapes	Actions numérotées, une par ligne dans la cellule
Résultat attendu	Ce qui doit se produire si l’application fonctionne
Chrome, Firefox, iOS, Android	Une colonne par plateforme, avec l’un des trois statuts : Not started, Passed, Failed

Chaque nouveau scénario démarre à Not started sur les quatre plateformes. Un Failed sur une plateforme peut créer un Bugfix dans le Backlog, avec la plateforme et le scénario en référence.

Périmètre de la version 1

- Gestion des utilisateurs et des rôles dans la console Keycloak ; pas d’écran d’administration dans Synapse.
- Notifications par e-mail uniquement.
- Menu du compte limité au profil et à la déconnexion.
- Tâche en attente d’une réponse humaine : elle reste en In progress avec la marque « en attente de ta réponse » et apparaît sur l’accueil dans « En attente de ta validation ». Pas de statut supplémentaire.
- Version 2 : écran d’administration de l’espace, paramètres du projet, notifications dans l’app, modèle local.